

OSSP WEEK 5 SKILL

Topic: To Implement Single Quote Handling for Literal String Preservation

1. Created a folder for week5 skill

```
ashborn@DESKTOP-I4G5Q4N:~$ mkdir week5_s
ashborn@DESKTOP-I4G5Q4N:~$ cd week5_s
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ pwd
/home/ashborn/week5_s
ashborn@DESKTOP-I4G5Q4N:~/week5_s$
```

2. Single quote vs Double quote check

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ name="Hasith"
ashborn@DESKTOP-I4G5Q4N:~/week5_s$
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo 'Hello $name'
Hello $name
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo "Hello $name"
Hello Hasith
```

3. Single quotes preserve literal content

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo '$HOME'
$HOME
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo '$USER'
$USER
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo 'Today is $(date)'
Today is $(date)
```

4. Store quoted strings in variables

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ message="OSSP Week 5 Skill Assignment"
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo "$message"
OSSP Week 5 Skill Assignment
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ message="Linux shell scripting is powerful"
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo "$message"
Linux shell scripting is powerful
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo '$message'
$message
```

5. Demonstrate spaces with double quotes

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ text="Linux Shell Programming"
ashborn@DESKTOP-I4G5Q4N:~/week5_s$
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ printf '<%s>\n' "$text"
<Linux Shell Programming>
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ printf '<%s>\n' $text
<Linux>
<Shell>
<Programming>
```

6. Argument Parsing

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ show_args() {
>     echo "Number of arguments: $#"
```

```
>     for arg in "$@"
>     do
>         echo "Argument: [$arg]"
>     done
>
>
> c
> ^C
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ show_args() {
>     echo "Number of arguments: $#"
```

```
>     for arg in "$@"; do
>         echo "Argument: [$arg]"
>     done
> }
```

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ show_args "$text"
Number of arguments: 1
Argument: [Linux Shell Programming]
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ show_args $text
Number of arguments: 3
Argument: [Linux]
Argument: [Shell]
Argument: [Programming]
```

7. Nested Tokens / Command Substitution

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo "Current user: $USER"
Current user: ashborn
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo "Current directory: $(pwd)"
Current directory: /home/ashborn/week5_s
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo "Current date: $(date)"
Current date: Thu Sep 17 16:42:08 UTC 2026
```

8. Quoted Command

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ command='echo "Hello from a quoted command"'
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ echo "$command"
echo "Hello from a quoted command"
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ bash -c "$command"
Hello from a quoted command
```

9. Parsing Validation

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ nano valid.sh
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ bash -n valid.sh
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ bash valid.sh
Hello Hasith
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ nano invalid.sh
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ nano invalid.sh
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ bash -n invalid.sh
invalid.sh: line 2: unexpected EOF while looking for matching `''
```

10. Final Combined Implementation

```
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ nano week5_quotes.sh
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ bash -n week5_quotes.sh
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ chmod +x week5_quotes.sh
ashborn@DESKTOP-I4G5Q4N:~/week5_s$ ./week5_quotes.sh
Single quotes: $name
Double quotes: Hasith
$HOME
$USER
<Linux Shell Programming>
<Linux>
<Shell>
<Programming>
Current user: ashborn
Current directory: /home/ashborn/week5_s
echo "Hello from a quoted command"
Hello from a quoted command
It's Linux
```

Summary:

In this practical, we learned how Bash handles single and double quotes while executing shell commands. We demonstrated literal string preservation, variable expansion, spaces in strings, argument parsing, and command substitution using commands such as `echo`, `printf`, and `$(...)`.

We also practiced quoted commands and syntax validation using `bash -n`. Finally, we created and executed a combined Bash script to verify all the concepts practically and checked the results through terminal outputs and screenshots.